

Chapter 3. Interacting with Kubernetes

As an application developer, you will want to interact with the Kubernetes cluster to manage objects that operate your application. Every call to the cluster is accepted and processed by the API server component. There are various ways to perform a call to the API server. For example, you can use a [web-based dashboard](#), a command-line tool like `kubectl`, or a direct HTTPS request to the RESTful API endpoints.

The exam does not test the use of a visual user interface for interacting with the Kubernetes cluster. Your only client for solving exam questions is `kubectl`. This chapter will touch on the Kubernetes API primitives and objects, as well as the different ways to manage objects with `kubectl`.

API Primitives and Objects

Kubernetes primitives are the basic building blocks anchored in the Kubernetes architecture for creating and operating an application on the platform. Even as a beginner to Kubernetes, you might have heard of the terms Pod, Deployment, and Service, all of which are Kubernetes primitives. There are many more that serve a dedicated purpose in the Kubernetes architecture.

To draw an analogy, think back to the concepts of object-oriented programming. In object-oriented programming languages, a class defines the blueprint of a real-world functionality: its properties and behavior. A Kubernetes primitive is the equivalent of a class. The instance of a class in object-oriented programming is an object, managing its own state and having the ability to communicate with other parts of the system. Whenever you create a Kubernetes object, you produce such an instance.

For example, a Pod in Kubernetes is the class of which there can be many instances with their own identity. Every Kubernetes object has a system-generated unique identifier (also known as UID) to clearly distinguish between the entities of a system. Later, we'll look at the properties of a Kubernetes object. [Figure 3-1](#) illustrates the relationship between a Kubernetes primitive and an object.

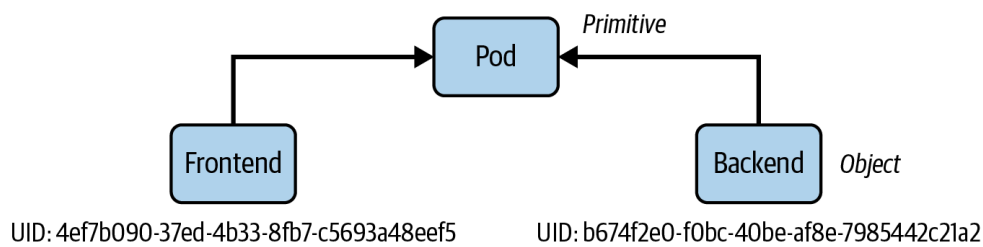


Figure 3-1. Kubernetes object identity

Every Kubernetes primitive follows a general structure, which you can observe if you look deeper at a manifest of an object, as shown in [Figure 3-2](#). The primary markup language used for a Kubernetes manifest is YAML.

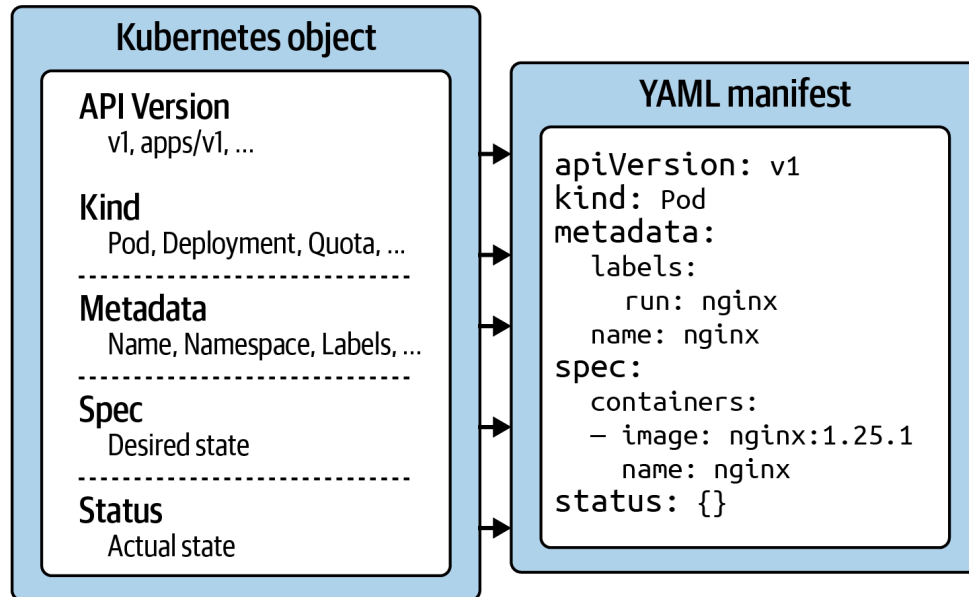


Figure 3-2. Kubernetes object structure

Let's look at each section and its relevance within the Kubernetes system:

API version

The Kubernetes API version defines the structure of a primitive and uses it to validate the correctness of the data. The API version serves a similar purpose as XML schemas to an XML document or JSON schemas to a JSON document. The version usually undergoes a maturity process—for example, from alpha to beta to final. Sometimes you see different prefixes separated by a slash (e.g., `apps`). You can list the API versions compatible with your cluster version by running the command `kubectl api-versions`.

Kind

The kind defines the type of primitive—e.g., a Pod or a Service. It ultimately answers the question, “What kind of resource are we dealing with here?”

Metadata

Metadata describes higher-level information about the object—e.g., its name, what namespace it lives in, or whether it defines labels and annotations. This section also defines the UID.

Spec

The specification (*spec* for short) declares the desired state—e.g., how should this object look after it has been created? Which image should run in the container, or which environment variables should be set?

Status

The status describes the actual state of an object. The Kubernetes controllers and their reconciliation loops constantly try to transition a Kubernetes object from the actual state into the desired state. The object has not yet been materialized if the YAML status shows the value `{}`.

With this basic structure in mind, let's look at how to create a Kubernetes object with the help of `kubectl`.

Using `kubectl`

`kubectl` is the primary tool for interacting with the Kubernetes clusters from the command line. The exam is exclusively focused on the use of `kubectl`. Therefore, it's paramount to understand its ins and outs and practice its use heavily.

This section provides you with a brief overview of its typical usage pattern. Let's start by looking at the syntax for running commands. A

`kubectl` execution consists of a command, a resource type, a resource name, and optional command line flags:

```
$ kubectl [command] [TYPE] [NAME] [flags]
```

The command specifies the operation you're planning to run. Typical commands are verbs like `create`, `get`, `describe`, or `delete`. Next, you'll need to provide the resource type you're working on, either as a full resource type or its short form. For example, you could work on a `service` here or use the short form, `svc`.

The name of the resource identifies the user-facing object identifier, effectively the value of `metadata.name` in the YAML representation. Be aware that the object name is not the same as the UID. The UID is an auto-generated, Kubernetes-internal object reference that you usually don't

have to interact with. The name of an object has to be unique across all objects of the same resource type within a namespace.

Finally, you can provide zero to many command-line flags to describe additional configuration behavior. A typical example of a command-line flag is the `--port` flag, which exposes a Pod's container port.

[Figure 3-3](#) shows a full `kubectl` command in action.

```
kubectl [command] [TYPE] [NAME] [flags]
         {      }   {    }   {   }   {  }
         get    pod  app  -o yaml
```

Figure 3-3. `kubectl` usage pattern

Over the course of this book, we'll explore the `kubectl` commands that will make you the most productive during the exam. There are many more, however, and they usually go beyond the ones you'd use on a day-to-day basis as an application developer. Next up, we'll have a deeper look at the `create` command, the imperative way to create a Kubernetes object. We'll also compare the imperative object creation approach with the declarative approach.

A crucial time-saving tool during the exam is `kubectl explain`, which provides instant access to resource specifications without needing to search documentation. For example, `kubectl explain pods.spec.containers` shows all available container configuration fields, while `kubectl explain deployment.spec.strategy.rollingUpdate` details rolling update parameters. This command is particularly valuable when you need to quickly verify field names, check if a property is a list or single value, or understand nested structures—use it liberally to avoid typos and save precious minutes that would otherwise be spent searching through Kubernetes documentation.

Managing Objects

You can create objects in a Kubernetes cluster in two ways: imperatively or declaratively. The following sections describe each approach, including their benefits, drawbacks, and use cases.

Imperative Object Management

Imperative object management does not require a manifest definition. You'll use `kubectl` to drive the creation, modification, and deletion of objects with a single command and one or many command-line options.

See the [Kubernetes documentation](#) for a more detailed description of imperative object management.

Creating objects

Use the `run` or `create` command to create an object on the fly. Any configuration needed at runtime is provided by command-line options. The benefit of this approach is the fast turnaround time without the need to wrestle with YAML structures. The following `run` command creates a Pod named `frontend` that executes the container image `nginx:1.29.0` in a container with the exposed port 80:

```
$ kubectl run frontend --image=nginx:1.29.0 --port=80
pod/frontend created
```

Updating objects

The configuration of live objects can still be modified. `kubectl` supports this use case by providing the `edit` and `patch` commands.

The `edit` command opens an editor with the raw configuration of the live object. Changes to the configuration will be applied to the live object after exiting the editor. The command will open the editor defined by the `KUBE_EDITOR`, or `EDITOR` environment variables, or fall back to `vi` for Linux or `notepad` for Windows. This command demonstrates the use of the `edit` command for the Pod live object named `frontend`:

```
$ kubectl edit pod frontend
```

The `patch` command allows for fine-grained modification of a live object on an attribute level using a JSON merge patch. The following example illustrates the use of `patch` command to update the container image tag assigned to the Pod created earlier. The `-p` flag defines the JSON structure used to modify the live object:

```
$ kubectl patch pod frontend -p '{"spec":{"containers":[{"name":"frontend","image":"nginx:1.29.2"}]}}'
pod/frontend patched
```

Deleting objects

You can delete a Kubernetes object at any time. During the exam, the need may arise if you made a mistake while solving a problem and want to start from scratch to ensure a clean slate. In a production Kubernetes environment, you'll want to delete objects that are no longer needed. The

following `delete` command deletes the Pod object by its name

`frontend`:

```
$ kubectl delete pod frontend  
pod "frontend" deleted
```

Upon execution of the `delete` command, Kubernetes tries to delete the targeted object gracefully so that there's minimal impact on the end user. If the object cannot be deleted within the default grace period (30 seconds), the kubelet attempts to forcefully terminate the object.

During the exam, end-user impact is not a concern. The most important goal is to complete all tasks in the time granted to the candidate. Therefore, waiting on an object to be deleted gracefully is a waste of time. You can force an immediate deletion of an object with the command-line option `--now`. The following command terminates the Pod named `nginx` using a `SIGKILL` signal:

```
$ kubectl delete pod nginx --now
```

Declarative Object Management

Declarative object management requires one or several manifests in the format of YAML or JSON describing the desired state of an object. You create, update, and delete objects using this approach.

The benefit of using the declarative method is reproducibility and improved maintenance, as the file is checked into version control in most cases. The declarative approach is the recommended way to create objects in production environments.

More information on declarative object management can be found in the [Kubernetes documentation](#).

Creating objects

The declarative approach creates objects from a manifest (in most cases, a YAML file) using the `apply` command. The command works by pointing to a file, a directory of files, or a file referenced by an HTTP(S) URL using the `-f` option. If one or more of the objects already exist, the command will synchronize the changes made to the configuration with the live object.

To demonstrate this functionality, we'll assume the following directories and configuration files. The following commands create objects from a

single file, from all files within a directory, and from all files in a directory recursively. Refer to files in the book's GitHub repository if you want to give it a try. Later chapters will explain the purpose of the primitives used here:

```
.
├── app-stack
│   ├── mysql-pod.yaml
│   ├── mysql-service.yaml
│   ├── web-app-pod.yaml
│   └── web-app-service.yaml
├── nginx-deployment.yaml
└── web-app
    ├── config
    │   ├── db-configmap.yaml
    │   └── db-secret.yaml
    └── web-app-pod.yaml
```

Creating an object from a single file:

```
$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
```

Creating objects from multiple files within a directory:

```
$ kubectl apply -f app-stack/
pod/mysql-db created
service/mysql-service created
pod/web-app created
service/web-app-service created
```

Creating objects from a recursive directory tree containing files:

```
$ kubectl apply -f web-app/ -R
configmap/db-config configured
secret/db-creds created
pod/web-app created
```

Creating objects from a file referenced by an HTTP(S) URL:

```
$ kubectl apply -f https://raw.githubusercontent.com/bmuschko/\
cka-study-guide/master/ch03/object-management/nginx-deployment.yaml
deployment.apps/nginx-deployment created
```

The `apply` command keeps track of the changes by adding or modifying the annotation with the key `kubectl.kubernetes.io/last-applied-configuration`. Here's an example of the annotation in the output of the `get pod` command:

```
$ kubectl get pod web-app -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"\
      "labels":{"app":"web-app"},"name":"web-app","namespace":"default"},
      "spec":{"containers":[{"envFrom":[{"configMapRef":{"name":"db-config"},
      {"secretRef":{"name":"db-creds"}}],"image":"bmuschko/web-app:1.0.1",
      "name":"web-app","ports":[{"containerPort":3000,"protocol":"TCP"}]}}]
      "restartPolicy":"Always"}}
...

```

Updating objects

Updating an existing object is done with the same `apply` command. All you need to do is to change the configuration file and then run the command against it.

KUBECTL CREATE VERSUS KUBECTL APPLY COMMAND

The `kubectl create` command is used to create new Kubernetes resources and will fail if the resource already exists, making it suitable for one-time resource creation. In contrast, `kubectl apply` uses a declarative approach that can both create new resources and update existing ones by comparing the desired state in your YAML/JSON file with the current state in the cluster, making it idempotent and ideal for managing resources over time.

[Example 3-1](#) modifies the existing configuration of a Deployment in the file *nginx-deployment.yaml*. I added a new label with the key `team` and changed the number of replicas from three to five.

Example 3-1. Modified configuration file for a Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
    team: red
spec:
```



```
    replicas: 5
...

```

The following command applies the changed configuration file. As a result, the number of Pods controlled by the underlying ReplicaSet is five:

```
$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment configured

```

The Deployment's `kubectl.kubernetes.io/last-applied-configuration` annotation reflects the latest change to the configuration:

```
$ kubectl get deployment nginx-deployment -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":
      "labels":{"app":"nginx","team":"red"},"name":"nginx-deployment", \
      "namespace":"default"},"spec":{"replicas":5,"selector":{"matchLabels
      {"app":"nginx"}},"template":{"metadata":{"labels":{"app":"nginx"}},'
      "spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx", \
      "ports":[{"containerPort":80}]}}}}}}
...

```

Deleting objects

While you can delete objects using the `apply` command by providing the options `--prune -l <labels>`, it is recommended to delete an object using the `delete` command and point it to the configuration file. The following command deletes a Deployment and the objects it controls (ReplicaSet and Pods):

```
$ kubectl delete -f nginx-deployment.yaml
deployment.apps "nginx-deployment" deleted

```

When you delete a Pod that's managed by a ReplicaSet or Deployment, the controller will automatically recreate it to maintain the desired replica count, making the deletion temporary and ineffective. Therefore, you should only modify or delete the resources you directly created (like the Deployment itself), rather than attempting to change the state of objects that are indirectly managed by controllers (like individual Pods created by that Deployment). This ensures your changes are permanent and

aligns with Kubernetes' declarative model where controllers continuously reconcile the actual state with the desired state.

You can use the `--now` option to forcefully delete Pods, as described in [“Deleting objects”](#).

Hybrid Approach

Sometimes you may want to go with a hybrid approach. You can start by using the imperative method to produce a manifest file without actually creating an object. You do so by executing the `run` or `create` command with the command-line options `-o yaml` and `--dry-run=client`:

```
$ kubectl run frontend --image=nginx:1.29.2 --port=80 \
-o yaml --dry-run=client > pod.yaml
```

You can now use the generate YAML manifest as a starting point to make further modifications before creating the object. Simply open the file with an editor, change the content, and execute the declarative `apply` command:

```
$ vim pod.yaml
$ kubectl apply -f pod.yaml
pod/frontend created
```

Which Approach to Use?

During the exam, using imperative commands is the most efficient and quickest way to manage objects. Not all configuration options are exposed through command-line flags, which may force you into using the declarative approach. The hybrid approach can help here.

GITOPS AND KUBERNETES

GitOps is a practice that leverages source code checked into Git repositories to automate infrastructure management, specifically in cloud native environments powered by Kubernetes. Tools such as [Argo CD](#) and [Flux](#) implement GitOps principles to deploy applications to Kubernetes through a declarative approach. Teams responsible for overseeing real-world Kubernetes clusters and the applications within them are highly likely to adopt the declarative approach.

While creating objects imperatively can optimize the turnaround time, in a real-world Kubernetes environment you'll most certainly want to use the declarative approach. A YAML manifest file represents the ultimate

source of truth of a Kubernetes object. Version-controlled files can be audited and shared, and they store a history of changes in case you need to revert to a previous revision.

Summary

Kubernetes represents its functionality for deploying and operating a cloud native application with the help of primitives. Each primitive follows a general structure: the API version, the kind, the metadata, and the desired state of the resources, also called the spec. Upon creation or modification of the object, the Kubernetes scheduler automatically tries to ensure that the actual state of the object follows the defined specification. Every live object can be inspected, edited, and deleted.

`kubectl` acts as a CLI-based client to interact with the Kubernetes cluster. You can use its commands and flags to manage Kubernetes objects. The imperative approach provides a fast turnaround time for managing objects with a single command, as long as you memorize the available flags. More complex configuration calls for the use of a YAML manifest to define a primitive. Use the declarative command to instantiate objects from that definition. The YAML manifest is usually checked into version control and offers a way to track changes to the configuration.